PlainML

Interpreter and Ahead-of-Time Compiler - README

CS-4031 Compiler Construction - Semester Project

Students: 23L-0603, 23L-0971, 23L-0708 Section: BCS-7H

Project overview

PlainML is a small domain-specific language for human-readable inference with fixed-weight, fully connected feed-forward neural networks. A PlainML source file names inputs, layers, neurons, weighted connections, biases, activation functions, and prediction values directly. The compiler does not call an external machine-learning framework during inference.

The project is implemented in two required phases. Phase 1 is an interpreter: Flex performs lexical analysis, Bison parses the program, and C code in the Bison implementation directly executes the network. Phase 2 modifies the compiler to generate LLVM IR. Clang then compiles the generated IR into a native executable, giving an Ahead-of-Time (AOT) compiler.

Final scope and changes from the proposal

The original proposal described a slightly broader Version 1. During implementation, the final scope was refined so the project stays focused on the required compiler pipeline and on neural-network inference.

- The proposed labels feature was deferred. Final PlainML reports numeric output directly instead of mapping 0/1 to text labels.
- The proposed explain command was deferred. Intermediate neuron calculations are still generated and executed, but they are not exposed through a separate language command.
- The proposal described a general abstract syntax tree. The final implementation uses domain-specific C structures for layers, neurons, connections, weights, biases, and prediction inputs. These structures serve as PlainML's internal representation for semantic analysis, interpretation, and LLVM generation.
- For a strict sequential feed-forward model, first-layer connections reference declared inputs and later layers reference neurons from the immediately previous layer.

Core language features

Construct	Meaning
network	Starts one named feed-forward network.
inputs	Declares scalar network inputs.
layer ... activation	Creates a layer using relu or step.
neuron	Declares a named neuron inside a layer.
weight	Connects an input/previous neuron with a fixed weight.
bias	Adds the neuron's constant bias.
predict ... with	Supplies input values for one prediction.

PlainML syntax and example

PlainML Version 1 uses one numeric type (double-precision floating point internally), fixed weights and biases, and two built-in activations: ReLU and step. A program contains one network definition followed by one predict statement.

Example program

```
network student_success {

    inputs {
        attendance;
        study_hours;
        assignments;
    }

    layer hidden activation relu {
        neuron effort {
            attendance weight 0.30;
            study_hours weight 0.80;
            assignments weight 0.60;
            bias -0.20;
        }
    }

    layer output activation step {
        neuron success {
            effort weight 1.00;
            bias -0.50;
        }
    }
}

predict student_success with {
    attendance = 0.82;
    study_hours = 0.60;
    assignments = 0.80;
}
```

How the example is evaluated

Each neuron computes a weighted sum, adds its bias, and then applies the layer activation:

```
weighted_sum = bias + sum(source_value * weight)

relu(x) = x when x > 0, otherwise 0
step(x) = 1 when x >= 0, otherwise 0
```

For the example above, effort = relu(0.82*0.30 + 0.60*0.80 + 0.80*0.60 - 0.20) = 1.006. The output neuron then computes step(1.006*1.00 - 0.50) = 1.

Semantic checking

- Every first-layer connection must reference a declared network input.
- Every later-layer connection must reference a neuron in the immediately previous layer.
- Duplicate input, layer, and neuron declarations are rejected in their scope.
- Prediction input names must exist and may only be supplied once.
- Every declared input must receive a prediction value.
- The predict statement must name the declared network.
- The grammar requires at least one input, one layer, one neuron per layer, one connection per neuron, and a bias.

Phase 1 - Interpreter

The Phase 1 interpreter is preserved on the Git branch interpreter-dev. It uses Flex and Bison only for the language front end and does not generate intermediate code. After parsing and semantic checking, the C functions in parser.y execute the network layer by layer.

Build Phase 1

```
git checkout interpreter-dev
make clean
make
```

Bison generates parser.tab.c and parser.tab.h from parser.y. Flex generates lex.yy.c from lexer.l. GCC then builds the interpreter executable named plainml.

Run Phase 1

```
./plainml < examples/test.plain
```

The interpreter parses the network, validates references and prediction inputs, executes the feed-forward calculation, and prints the final prediction.

```
Prediction:
  success = 1.0000
```

Phase 1 execution flow

```
PlainML source
    -> Flex tokens
    -> Bison grammar/actions
    -> C network structures
    -> semantic checks
    -> direct C execution
    -> prediction
```

Phase 2 - AOT compiler

The final branch modifies the Bison implementation so that a successful predict statement generates LLVM IR in output.ll. The generated IR contains floating-point multiplication/addition, comparisons, select instructions for activations, and printf calls for output. Clang compiles this IR into a native executable named plainml_aot.

Build one PlainML program with AOT

```
make clean
make aot SOURCE=examples/test.plain
```

Run the native executable

```
make run-aot

Prediction:
  success = 1.000000
```

Phase 2 compilation flow

```
PlainML source
    -> Flex
    -> Bison + semantic checks
    -> LLVM IR (output.ll)
    -> Clang
    -> native executable (plainml_aot)
    -> prediction
```

Linux setup and dependencies

PlainML is designed and tested for Linux. The development environment used Ubuntu under WSL, but the build commands are standard Linux commands.

Required dependencies

Tool	Purpose
Flex	Lexical analysis and lex.yy.c generation.
Bison	Parsing and parser.tab.c/parser.tab.h generation.
GCC	Builds the PlainML compiler/interpreter front end.
Clang / LLVM	Compiles generated LLVM IR into a native executable.
GNU Make	Provides repeatable build and test targets.
libfl	Flex runtime library used by the GCC link command.

Ubuntu installation

```
sudo apt update
sudo apt install flex bison gcc make clang libfl-dev
```

Verify the toolchain

```
flex --version
bison --version
gcc --version
make --version
clang --version
```

Normal final-branch workflow

From the repository root:

```
make clean
make aot SOURCE=examples/test.plain
make run-aot
```

To compile another PlainML program, replace SOURCE with another .plain file, for example:

```
make aot SOURCE=examples/deep_network.plain
make run-aot
```

Automated tests

The final Makefile provides one command for the complete regression suite:

```
make clean
make test
```

The valid AOT tests include the basic network, a multi-neuron network, a deeper network, and the pretrained student-success network. The error suite verifies that invalid programs are rejected.

Expected semantic-error cases

- Unknown input referenced by a first-layer neuron.
- Missing prediction input value.
- Duplicate prediction input value.
- Duplicate input declaration.
- Duplicate layer declaration.
- Duplicate neuron declaration in one layer.

Clang may print a warning that it is overriding the module target triple with the host Linux target. This is harmless; the generated executable still compiles and runs correctly.

Pretrained student-success demonstration

A separate example demonstrates that PlainML can execute real learned neural-network parameters instead of only hand-written weights. The model is trained offline and then exported into ordinary PlainML source. The final AOT executable performs inference without Python, scikit-learn, TensorFlow, PyTorch, or another ML runtime.

Dataset

The demonstration uses the UCI Machine Learning Repository Student Performance dataset (Portuguese-language course). It contains 649 student records collected from two Portuguese secondary schools. The target is the final grade G3 (0-20).

Dataset citation: Paulo Cortez, Student Performance, UCI Machine Learning Repository, 2008. DOI: 10.24432/C5TG7T. The dataset is distributed under CC BY 4.0.

Prediction task

For this demonstration, success is defined as:

```
success = 1 when G3 >= 10
success = 0 when G3 < 10
```

G3 is not supplied as an input. The network uses:

PlainML input	Dataset meaning
first_grade	G1 - first-period grade
second_grade	G2 - second-period grade
study_time	Weekly study-time category
past_failures	Number of previous class failures
absences	School absences
wants_higher_education	Whether the student wants higher education (0/1)
school_support	Extra educational support (0/1)
family_support	Family educational support (0/1)

Model architecture and measured result

The exported model has 8 inputs, one hidden layer with 6 ReLU neurons, and one step output neuron. The training script uses a stratified 75/25 split, scales the training features, balances only the training portion by oversampling the minority class, and trains an MLPClassifier. Standardization is algebraically folded into the first-layer weights and biases before export.

Metric	Result
Dataset rows	649
Training rows	486
Held-out test rows	163
Held-out accuracy	85.89%
Balanced accuracy	78.57%
True negatives / False positives	17 / 8
False negatives / True positives	15 / 123
Export equivalence check	PASS

Using the pretrained model

The exported weights are already stored in examples/pretrained_student_success.plain, so no Python packages or dataset download are required to compile and run it.

Compile and run

```
make clean
make aot SOURCE=examples/pretrained_student_success.plain
make run-aot
```

Included held-out demonstration

Input	Value
first_grade	15
second_grade	16
study_time	2
past_failures	0
absences	2
wants_higher_education	1
school_support	0
family_support	1
Actual final grade G3	16
Expected class	1
PlainML prediction	1

The resulting native executable prints:

```
Prediction:
  success = 1.000000
```

Optional: reproduce the training/export

The training script is included only to show where the pretrained weights came from. It is not required by the PlainML compiler or by the AOT executable.

Optional Python dependencies

```
sudo apt install python3 python3-venv python3-pip
python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy pandas scikit-learn
```

Download and extract the UCI dataset

```
  mkdir -p data
  curl -L \
    'https://archive.ics.uci.edu/static/public/320/student%2Bperformance.zip' \
    -o data/student-performance.zip

  python3 -m zipfile -e \
    data/student-performance.zip \
    data/student-performance

  python3 -m zipfile -e \
    data/student-performance/student.zip \
    data/student-performance/student
```

Retrain and export

```
python tools/train_student_model.py     data/student-performance/student/student-por.csv
```

The script prints held-out metrics, verifies that the exported raw-input mathematics exactly matches the scikit-learn predictions, and rewrites examples/pretrained_student_success.plain with the learned weights. The data/ directory and .venv/ are intentionally ignored by Git.

Important: this pretrained model is an educational demonstration of compiler-generated inference. It should not be used to make real academic decisions about students.

Project files

File	Description
.gitignore	Ignores generated parser/lexer files, executables, LLVM output, local IDE files, optional training environment, and downloaded data.
Makefile	Build targets for the compiler, AOT compilation, execution, regression tests, semantic-error tests, and cleanup.
lexer.l	Flex rules for PlainML keywords, identifiers, numbers, punctuation, whitespace, and lexical fallback.
parser.y	Bison grammar plus C structures, semantic checking, Phase 1 execution logic, and Phase 2 LLVM IR generation.
README.pdf	This Linux build/run/use/dependency/file documentation.
README.md	Minimal repository landing page.
examples/test.plain	Canonical two-layer student-success example.
examples/multi_neuron.plain	Valid network with multiple hidden neurons and multiple incoming connections.
examples/deep_network.plain	Valid three-layer network used to test deeper AOT generation.
examples/pretrained_student_success.plain	Exported real trained student pass/fail model with learned weights.
examples/errors/unknown_input.plain	Invalid program: first-layer source is not a declared input.
examples/errors/missing_prediction_input.plain	Invalid program: a declared input has no prediction value.
examples/errors/duplicate_prediction_input.plain	Invalid program: one prediction input is supplied twice.
examples/errors/duplicate_input_decl.plain	Invalid program: duplicate input declaration.
examples/errors/duplicate_layer.plain	Invalid program: duplicate layer declaration.
examples/errors/duplicate_neuron.plain	Invalid program: duplicate neuron declaration in one layer.
tools/train_student_model.py	Optional reproducibility tool that trains the demonstration model and exports PlainML weights.

Generated files (not committed)

Generated file	Purpose
parser.tab.c / parser.tab.h	Generated by Bison.
lex.yy.c	Generated by Flex.
plainml	Compiled PlainML front-end / interpreter executable.
output.ll	LLVM IR generated from the selected PlainML program.
plainml_aot	Native executable produced by Clang from output.ll.

Final limitations

PlainML Version 1 is intentionally limited to fixed-weight, sequential feed-forward inference with scalar numeric inputs and ReLU/step activations. It does not implement training, backpropagation, datasets as a language feature, tensors, convolutional/recurrent/transformer layers, GPU execution, general-purpose loops, or user-defined functions. Training the optional demonstration model happens outside PlainML; only the learned parameters are compiled by PlainML.